

Trabajo Final-Laura Cerda

Informe - Sistema de Gestión de Fondos para una DAO-

El contrato inteligente DAOFund está diseñado para administrar fondos de una organización de forma descentralizada, utilizando diferentes roles y un sistema de votación para aprobar los gastos. Para almacenar la información necesaria utiliza variables, estructuras (struct), enumeraciones (enum), mappings y listas de direcciones.

La variable owner almacena la dirección de la cuenta que desplegó el contrato. Esta dirección se establece en el constructor y se declara como immutable, por lo que no puede modificarse posteriormente. El contrato también utiliza los mappings partners y spenders para determinar qué direcciones tienen actualmente los roles de Partner y Spender. Además, existen listas que almacenan las direcciones de los Partners y Spenders registrados.

Para las solicitudes de gastos se utiliza la estructura SolicitudGasto. Esta estructura contiene la dirección de destino del pago, el monto solicitado, el plazo límite para votar, la cantidad de aprobaciones necesarias, la cantidad de aprobaciones recibidas y el estado de la solicitud. También contiene mappings internos que permiten registrar qué direcciones están obligadas a votar y cuáles ya emitieron su voto. De esta manera, cada participante puede votar una sola vez en cada solicitud.

El contrato también utiliza la enumeración SolicitudGasto, que permite representar tres estados posibles de una solicitud: Pendiente, cuando todavía está pendiente; Ejecutada, cuando fue aprobada y ejecutada; y Rechazada, cuando fue rechazada o expiró. Además, la variable reservedFunds permite llevar un control de los fondos que ya están reservados para solicitudes de gasto pendientes.

Entre las funciones principales se encuentra enrollAsPartner(), que permite que una wallet se registre como Partner enviando exactamente 0,4 ETH. La dirección se obtiene automáticamente mediante msg.sender, por lo que no es necesario indicar la dirección como parámetro. También se encuentra PrimerSpenderDesignado (), función que permite al Owner designar al primer Spender del contrato.

El contrato incluye además funciones relacionadas con la promoción de Partners a Spenders. Un Partner puede solicitar la promoción mediante SpenderPromocionSolicitada (), y el Owner puede aprobarla o rechazarla mediante SpenderPromovido () o SpenderPromovRechazado ().

La gestión de gastos se realiza principalmente mediante SolicitudGastoCreado(). Un Spender puede crear una solicitud indicando una wallet de destino y un monto. Al crearla, el contrato establece un plazo de 24 horas y registra qué participantes deben votar. Las funciones GastoEjecutado () y SolicitudGastorechazada() permiten gestionar posteriormente la solicitud. Para evitar que una misma wallet vote dos veces cuando posee más de un rol, el contrato utiliza un único registro de votación por dirección.

Finalmente, RechazarSolicitudCaduco () permite marcar como rechazada una solicitud cuyo plazo de 24 horas ya terminó, mientras que removeMiembro() permite al Owner eliminar a un Partner o Spender. El contrato también dispone de varias funciones de

consulta (view) para obtener información sobre los miembros, las solicitudes y el estado de los fondos sin modificar el estado de la blockchain.

1. Objetivo

El contrato implementa una DAO simplificada para administrar un fondo común en ETH. Ningún actor puede disponer individualmente de los fondos: toda salida requiere aprobación unánime de los votantes definidos para la solicitud.

2. Roles

****Owner:**** wallet que despliega el contrato. Se guarda como ``immutable``. Puede designar al primer Spender, aprobar/rechazar promociones y expulsar miembros. No es Partner ni Spender.

****Partner:**** ingresa aportando exactamente 0.4 ETH, puede votar y puede solicitar convertirse en Spender.

****Spender:**** puede crear solicitudes de gasto y votar. Puede ser simultáneamente Partner; en ese caso tiene un solo voto.

3. Enrollment

Se exige exactamente ``0.4 ether``. Si se envía menos o mas, el ``require`` revierte la transacción. Se eligió esta regla porque evita definir una lógica adicional para devolver o contabilizar excedentes. El monto aceptado queda en el fondo común.

4. Solicitud de gasto

Un Spender indica destino y monto. El destino no puede ser ``address(0)``, Owner, Partner ni el propio Spender. La solicitud tiene 24 horas de vigencia.

5. Snapshot de votantes

El conjunto de votantes se congela al crear la solicitud. Un Partner incorporado despues no vota esa solicitud. Una expulsion posterior tampoco modifica sus requisitos. Esto mantiene reglas estables durante toda la vida de una solicitud.

El snapshot se almacena mediante ``mapping(address => bool) requiredVoter`` dentro de cada ``ExpenseRequest``.

6. Votacion

Cada solicitud almacena ``requiredApprovals`` y ``approvals``. Cada wallet puede votar una sola vez mediante ``mapping(address => bool) voted``. La ejecucion se produce cuando ``approvals == requiredApprovals``.

No existe voto negativo separado: si no se alcanza unanimidad dentro de 24 horas, la solicitud puede marcarse como ``Rejected`` mediante `'RechazarSolicitudCaduco'`.

7. Expiracion

Ethereum no ejecuta codigo automaticamente cuando se cumple un plazo. Por eso el vencimiento se comprueba al interactuar y cualquier usuario puede llamar ``RechazarSolicitudCaduco(requestId)`` despues del deadline.

8. Reintentos

Una solicitud ejecutada o rechazada no puede reabrirse. Para repetir un gasto debe crearse una nueva solicitud, preservando un historial claro.

9. Expulsion

El Owner puede expulsar a un Partner, Spender o wallet con ambos roles. La membresia actual se actualiza, pero las solicitudes existentes conservan el snapshot tomado al crearse. El enrollment no se devuelve: se considera una contribucion ya incorporada al fondo.

10. Reserva de fondos

Se utiliza `reservedFunds`. Al crear una solicitud, el monto queda reservado para evitar que varias solicitudes pendientes comprometan mas ETH que el disponible. La reserva se libera al ejecutar o rechazar por vencimiento.

11. Estructuras y requisitos técnicos

- `mapping(address => bool)` para Partners y Spenders.
- `address[]` para poder enumerar miembros.
- `SolicitudGasto` para el estado de cada gasto.
- Funciones de escritura: enrollment, designacion, promocion, solicitud, voto, expiracion y expulsion.
- Funciones `view` para roles, balance, solicitudes y listas.
- Modifiers: `SoloOwner`, `SoloPartner`, `SoloSpender`.
- Eventos para enrollment, solicitudes, votos, ejecuciones, promociones y expulsiones.
- Visibilidad explicita: `external`, `external view`, `internal`, `public immutable`.

12. Seguridad basica

Antes de transferir ETH se actualiza el estado y se libera la reserva, siguiendo checks-effects-interactions. Tambien se comprueba el balance disponible y se evita sobre-reservar fondos.

13. Limitaciones

Es una implementacion educativa. Construir snapshots recorriendo arrays tiene un costo que crece con la cantidad historica de miembros

14. Flujo

Seed phrase propia

↓

MetaMask

↓

Cuenta propia

↓

Red Sepolia

↓

Deploy de DAOFund.sol

↓

Transacción firmada por tu cuenta

↓

Contrato publicado en Sepolia

↓

Hash de transacción + dirección del contrato

una vez creada una solicitud, el conjunto de votantes queda congelado (snapshot). Si posteriormente expulsan a un miembro, su situación respecto de esa solicitud no cambia: si ya votó, su voto sigue contando; si no votó, sigue siendo necesario para esa solicitud. Esto mantiene las reglas de una solicitud inmutables y evita tener que recorrer todas las solicitudes abiertas cada vez que se expulsa a alguien.

El contrato es funcional, pero hay una cuestión de **gas** que se menciona a continuación

Al crear una solicitud hacemos:

```
for (...) {  
    // recorrer Partners  
}
```

```
for (...) {  
    // recorrer Spenders  
}
```

Esto es perfectamente válido para una **implementación educativa**, pero en una DAO real podría resultar costoso si existen miles de miembros.

La razón es que Solidity no permite obtener directamente todos los elementos de:

```
mapping(address => bool)
```

Por eso agregamos:

```
address[] private partnerList;
```

```
address[] private spenderList;
```

Decisiones que implementa este contrato

Regla	Implementación
Owner	immutable, establecido en constructor
Owner es Partner	✗
Owner es Spender	✗
Enrollment	Exactamente 0.4 ether
Exceso de ETH	✗ Revert
Falta de ETH	✗ Revert
Partner	Puede votar
Spender	Puede crear gastos y votar
Spender + Partner	Permitido
Doble voto por tener dos roles	✗
Primer Spender	Designado por Owner
Promoción posterior	Partner solicita + Owner aprueba
Destino = Owner	✗
Destino = Partner	✗
Destino = propio Spender	✗
Destino = otro Spender	✓
Duración solicitud	24 horas
Unanimidad	Sí
Nuevo Partner después de crear solicitud	No participa en esa solicitud
Expulsión después de crear solicitud	No modifica el snapshot
Voto duplicado	✗
Solicitud vencida	Rechazada
Reintento de solicitud	Nueva solicitud

Regla	Implementación
Enrollment al expulsar	Permanece en el fondo
Ejecución	Solo después de unanimidad
Evento de ejecución	GastoSolicitado
Lectura de roles	view
Lectura de solicitudes	view
Control de acceso	SoloOwner, SoloPartner, SoloSpender

LINK: [Address: 0x5B38Da6a...f56beddC4 | Etherscan Sepolia](#)

[Address: 0x5B38Da6a...f56beddC4 | Etherscan Sepolia](#)